

RAG Data Pipeline — Multi-Modal Ingestion Extension

Low-Level Design (LLD) Document

Component: Image / Audio / Video Pre-processing for the existing RAG Ingestion

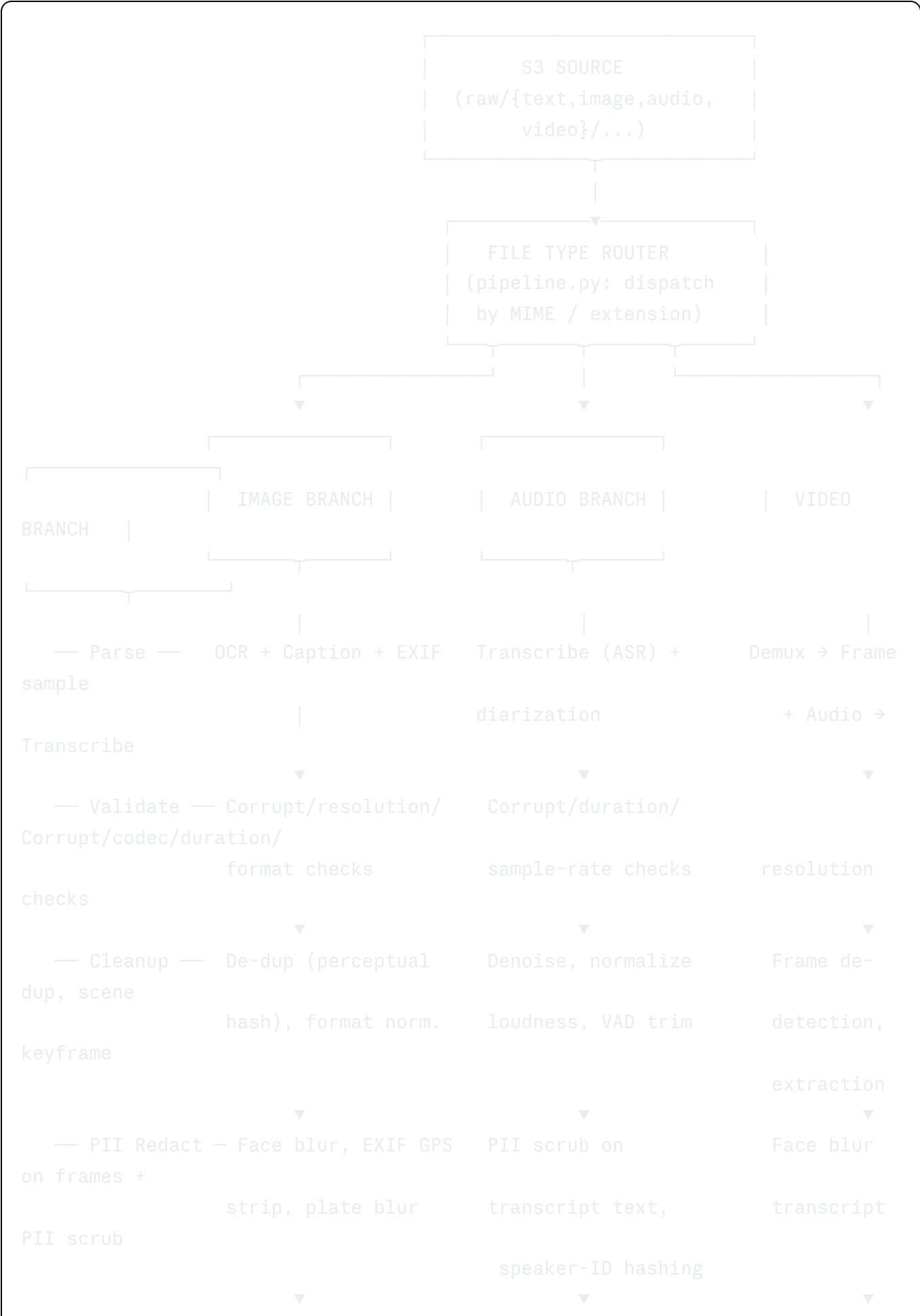
Framework Base architecture (unchanged): (S3 Source → Parse → Validate → Cleanup → PII Redact → Chunk → Tag Metadata → Target (S3 Vectors / OpenSearch)) **Goal:**

Extend the current text-only pipeline to natively support (image), (audio), and (video) documents using the same orchestration, config, and manifest patterns already in the repo — no architectural rewrite, only new modality handlers plugged into existing modules.

1. Design Principles

1. **One pipeline, many modalities.** (pipeline.py) stays the single orchestrator. It routes by file type, not by a separate pipeline per modality.
 2. **Strategy pattern for modality-specific logic.** Every stage ((Parse), (Cleanup), (PII Redact), (Chunk)) gets a (TEXT) / (IMAGE) / (AUDIO) / (VIDEO) strategy implementation behind a common interface. Existing text logic becomes (TextStrategy), so nothing already in production breaks.
 3. **Fail isolated, not global.** A bad video shouldn't halt the batch — same dead-letter/quarantine pattern your (delete_vectors_manifest.py) / manifest-driven design already implies.
 4. **Metadata schema is additive.** Existing metadata fields stay; modality-specific fields are added as a nested block, so downstream consumers ((index_manager.py), retrieval) don't break.
 5. **Embeddings are modality-aware but co-located.** All vectors — text, image, audio-transcript, video-frame — land in the **same S3 Vectors index/bucket**, distinguished by (modality) metadata, so retrieval can do unified or filtered search.
-

2. Extended Architecture





Quarantine/failure path (applies to every branch, same as your current manifest-driven cleanup): failed items are written to `s3://.../quarantine/{modality}/` and logged to a manifest, mirroring `delete_vectors_manifest.py`'s pattern, so retries and audits reuse existing tooling.

3. Module-Level Changes (mapped to your existing repo)

Existing File	Change	New Responsibility
<code>config.py</code>	Extend	Add <code>SUPPORTED_MODALITIES</code> , per-modality size/duration limits, model endpoints (ASR, OCR, embedding models), chunking window sizes
<code>parsers.py</code>	Extend	Add <code>ImageParser</code> , <code>AudioParser</code> , <code>VideoParser</code> classes implementing common <code>BaseParser</code> interface
<code>validators.py</code>	Extend	Add modality-specific validation rules (resolution floor, codec allow-list, duration ceiling, sample-rate check)
<code>cleanup.py</code>	Extend	Add <code>ImageCleaner</code> (perceptual-hash dedup, format normalization to PNG/JPEG), <code>AudioCleaner</code> (loudness normalization, silence trim), <code>VideoCleaner</code> (scene detection, keyframe extraction)
<code>privacy.py</code>	Extend	Add <code>FaceRedactor</code> (image/video frames), <code>PlateRedactor</code> , <code>EXIFStripper</code> , extend existing PII-regex redaction to run on ASR transcript output
<code>chunker.py</code>	Extend	Add <code>ImageChunker</code> (whole-image or tile-grid for very large images), <code>AudioChunker</code> (fixed time-window w/ overlap), <code>VideoChunker</code> (scene/shot-aligned, joins keyframe + transcript window)
<code>metadata.py</code>	Extend	Add nested schema block per modality (see §5) while keeping existing text fields untouched
<code>index_manager.py</code>	Extend	Route each chunk to the correct embedding model based on <code>modality</code> , write to shared S3 Vectors index with <code>modality</code> as a filterable metadata field
<code>index_examples.py</code>	Extend	Add example calls for image/audio/video ingestion + filtered retrieval by modality
<code>evaluate.py</code>	Extend	Add modality-specific quality checks (OCR confidence threshold, ASR word-error-rate sampling, blur/face-detection recall spot-check)
<code>pipeline.py</code>	Extend	Add file-type router at entry; dispatch to modality-specific strategy set; same orchestration loop

Existing File	Change	New Responsibility
New: <code>media_utils.py</code>	New	Shared low-level helpers: frame extraction (ffmpeg wrapper), audio demux, perceptual hashing, format conversion — used by parsers/cleanup across image/audio/video
New: <code>embeddings.py</code>	New	Thin abstraction over modality-specific embedding calls (text embedding model, image embedding model, audio/transcript embedding model) so <code>index_manager.py</code> doesn't need model-specific branching logic inline

No file is deleted or renamed — this is additive, matching your "reusable framework" description in the README.

4. Class Design (interfaces)

```
python
```

```

# parsers.py
class BaseParser(ABC):
    @abstractmethod
    def parse(self, s3_key: str) -> ParsedDocument: ...

class TextParser(BaseParser):    # existing behavior, wrapped
    ...

class ImageParser(BaseParser):
    """
    - Load image (Pillow / OpenCV)
    - Extract EXIF metadata
    - Run OCR (e.g. Tesseract / Textract) -> embedded text
    - Run captioning model -> semantic description (optional, config-gated)
    - Output: ParsedDocument(content=caption+ocr_text, raw_ref=s3_key, media_type="image")
    """

class AudioParser(BaseParser):
    """
    - Extract audio metadata (duration, sample rate, channels)
    - Run ASR (e.g. Amazon Transcribe / Whisper) -> transcript with timestamps
    - Optional speaker diarization -> speaker-labeled segments
    - Output: ParsedDocument(content=transcript, segments=[...], media_type="audio")
    """

class VideoParser(BaseParser):
    """
    - Demux via ffmpeg: extract audio track + sampled frames (e.g. 1 fps or scene changes)
    - Audio track -> AudioParser.parse()
    - Frames -> scene-change detection -> keyframes
    - Optional: per-keyframe captioning
    - Output: ParsedDocument(content=transcript, keyframes=[...], media_type="video")
    """

```

python

```

# chunker.py
class BaseChunker(ABC):
    @abstractmethod
    def chunk(self, doc: ParsedDocument) -> list[Chunk]: ...

class ImageChunker(BaseChunker):
    """Default: 1 image -> 1 chunk (image embedding + OCR/caption text).
       Large images (> config.IMAGE_TILE_THRESHOLD_PX) -> tile grid, each tile
       with positional metadata (row, col)."""

class AudioChunker(BaseChunker):
    """Fixed time-window chunks (default 30s, 5s overlap), each chunk carries
       start_ts/end_ts and the transcript slice for that window."""

class VideoChunker(BaseChunker):
    """Scene/shot-based: one chunk per detected scene, combining the scene's
       keyframe embedding + the time-aligned transcript slice.
       Falls back to fixed time-window if scene detection finds < 2 scenes."""

```

python

```

# privacy.py additions
class FaceRedactor:
    """Detect faces (e.g. AWS Rekognition / MediaPipe) and blur/pixelate in-place
       before the media is persisted to the processed bucket."""

class EXIFStripper:
    """Strip GPS + device identifiers from image/video metadata."""

# existing PIIRedactor (regex/NER based) is reused unmodified on ASR transcript

```

python

```
# embeddings.py (new)
class EmbeddingRouter:
    """
    modality -> model mapping, e.g.:
    text      -> Titan Text Embeddings / Cohere
    image     -> Titan Multimodal Embeddings / CLIP
    audio/video -> transcript text through the same text embedding model
                    (keeps vector space unified for cross-modal search),
                    optionally + a separate audio-embedding model for
                    acoustic similarity search if required
    """
    def embed(self, chunk: Chunk) -> list[float]: ...
```

5. Metadata Schema (additive)

```
json
{
  "doc_id": "uuid",
  "modality": "image | audio | video | text",
  "source_s3_key": "raw/video/clip123.mp4",
  "processed_s3_key": "processed/video/clip123/",
  "ingestion_timestamp": "iso8601",
  "chunk_id": "uuid",
  "pii_redacted": true,

  "image_meta": {
    "width": 1920, "height": 1080, "format": "jpeg",
    "ocr_text_present": true, "caption": "string",
    "perceptual_hash": "string", "exif_stripped": true
  },
  "audio_meta": {
    "duration_sec": 184.2, "sample_rate": 16000,
    "language": "en", "speaker_count": 2,
    "transcript_confidence": 0.93
  },
  "video_meta": {
    "duration_sec": 612.0, "fps": 30, "resolution": "1920x1080",
    "scene_count": 14, "scene_index": 3,
    "keyframe_hash": "string", "audio_transcript_ref": "chunk_id"
  }
}
```


Only the block relevant to the chunk's modality is populated; text chunks keep the existing flat schema you already have in `metadata.py`.

6. Validation Rules (validators.py additions)

Modality	Checks
Image	Format allow-list (jpg/png/tiff/webp), min resolution, max file size, corrupt-file check (attempt decode), NSFW/violence classifier (optional, config-gated)
Audio	Format allow-list (mp3/wav/flac/m4a), max duration, min sample rate, silence-only detection (reject if >95% silence)
Video	Codec allow-list (h264/h265/vp9), max duration, max resolution, corrupt-container check (ffprobe), frame-rate sanity check

Failures route to the same quarantine + manifest pattern used today — no new failure-handling mechanism needed.

7. Storage Layout (S3)

```
s3://<bucket>/raw/{text|image|audio|video}/...
s3://<bucket>/processed/{modality}/{doc_id}/
    ├── redacted_original.*          (post PII-redaction media)
    ├── keyframes/ (video only)
    ├── transcript.json (audio/video only)
    └── chunks/{chunk_id}.json
s3://<bucket>/quarantine/{modality}/...
s3://vectors-bucket/index/<index_name>/ (single shared index, filter by
`modality`)
```

8. Sequence (single video file, illustrative)

- `pipeline.py` detects `.mp4` extension → routes to Video branch.
- `VideoParser.parse()` → ffmpeg demux → audio.wav + sampled frames.
- Audio → `AudioParser` (nested call) → ASR transcript with timestamps.
- Frames → scene-change detection → keyframe list.
- `validators.py` → codec/duration/resolution checks. Fail → quarantine + manifest entry, stop.
- `cleanup.py` → keyframe de-dup, scene boundary refinement.

7. `privacy.py` → `FaceRedactor` on keyframes, `EXIFStripper`, PII-regex on transcript text.
 8. `chunker.py` (`VideoChunker`) → one chunk per scene (keyframe + transcript slice).
 9. `metadata.py` → attach `video_meta` block per chunk.
 10. `embeddings.py` → embed keyframe (image model) + transcript slice (text model); combine or store as dual vectors per chunk (config-driven: `single_fused_vector` vs `dual_vector`).
 11. `index_manager.py` → write to S3 Vectors index with `modality: "video"`.
-

9. Config Additions (config.py)

python

```
SUPPORTED_MODALITIES = ["text", "image", "audio", "video"]

IMAGE_MAX_SIZE_MB = 25
IMAGE_TILE_THRESHOLD_PX = 4096
IMAGE_ALLOWED_FORMATS = ["jpeg", "png", "tiff", "webp"]

AUDIO_MAX_DURATION_SEC = 3600
AUDIO_CHUNK_WINDOW_SEC = 30
AUDIO_CHUNK_OVERLAP_SEC = 5
AUDIO_ALLOWED_FORMATS = ["mp3", "wav", "flac", "m4a"]

VIDEO_MAX_DURATION_SEC = 7200
VIDEO_FRAME_SAMPLE_FPS = 1
VIDEO_ALLOWED_CODECS = ["h264", "h265", "vp9"]

EMBEDDING_STRATEGY = "dual_vector"    # or "single_fused_vector"

ASR_MODEL_ENDPOINT = "<amazon-transcribe | whisper endpoint>"
OCR_MODEL_ENDPOINT = "<textract | tesseract>"
IMAGE_EMBEDDING_MODEL = "<titan-multimodal | clip endpoint>"
FACE_REDACTION_MODEL = "<rekognition | mediapipe>"
```

10. Open Design Decisions (need your input before implementation)

1. **Embedding fusion:** for video/audio, do you want one fused vector per chunk (image+text averaged/concatenated) or two separate vectors (`dual_vector`) retrievable independently? This affects `embeddings.py` and index schema.
2. **ASR/OCR/embedding model choice:** AWS-native (Transcribe, Textract, Titan Multimodal, Rekognition) vs open-source (Whisper, Tesseract, CLIP) — affects cost,

latency, and whether everything stays in-VPC.

3. **Captioning for images/video keyframes:** optional but improves text-searchability — adds a model call per image/keyframe, so it's a cost/quality tradeoff worth gating behind config.
 4. **Face/plate redaction scope:** blur-in-place before storage (safer, irreversible) vs store both original (access-controlled) and redacted copy — compliance-driven decision.
-

This document assumes ffmpeg, an OCR engine, and an ASR engine are available in the pipeline's runtime environment (Lambda layer, container, or Glue job) — let me know your compute target (Lambda / ECS / Glue / EC2) and I can size the runtime, memory, and timeout requirements for the video branch specifically, since frame extraction and ASR are the most resource-intensive stages.